

Redis Cache Implementation (Cache Aside Pattern)

Step 1 : Client Requests Inventory

The client requests inventory details from the Inventory Service.

API

```
GET /inventory/Laptop
```

Flow

```
Client
|
v
Inventory Service
```

Explanation

The request first reaches the Inventory Service.

The Inventory Service does **not** directly access PostgreSQL.

It first checks Redis because Redis is much faster than a relational database.

Step 2 : Check Redis Cache

The Inventory Service first checks whether the requested inventory already exists in Redis.

Code

```
public Inventory getInventory(String product) {

    String key = CACHE_PREFIX + product;

    Inventory inventory =
        redisTemplate.opsForValue().get(key);
```

```
    if (inventory != null) {  
        log.info("CACHE HIT");  
        return inventory;  
    }  
}
```

Explanation

Redis is always checked before PostgreSQL.

If data exists in Redis, it is immediately returned.

Average lookup time is **O(1)**.

Flow

```
Inventory Service  
    |  
    |  
Check Redis Cache
```

Step 3 : Cache HIT

Suppose Redis already contains the product.

Redis

```
Key  
  
inventory::Laptop  
  
Value  
  
{  
    "product": "Laptop",  
    "stock": 95  
}
```

Code

```
Inventory inventory =  
    redisTemplate.opsForValue().get(key);  
  
if (inventory != null) {  
    return inventory;  
}
```

Logs

```
=====
```

Checking Redis Cache

CACHE HIT

Returning Laptop from Redis

```
=====
```

Flow

```
Client  
  |  
Inventory Service  
  |  
Redis  
  |  
Inventory Returned
```

No PostgreSQL query is executed.

Step 4 : Cache MISS

Suppose Redis does not contain the requested product.

Redis

```
inventory::Laptop
```

```
NOT FOUND
```

Inventory Service fetches data from PostgreSQL.

Code

```
Inventory inventory =  
    repository.findById(product)  
        .orElseThrow(() ->  
            new RuntimeException(  
                "Product not found"));
```

Logs

```
=====
```

Checking Redis Cache

CACHE MISS

Loading Laptop from PostgreSQL

```
=====
```

Flow

```
Client  
|  
Inventory Service  
|  
Redis  
|  
MISS  
|  
PostgreSQL
```

Step 5 : Save Inventory into Redis

After retrieving inventory from PostgreSQL, save it into Redis.

Code

```
redisTemplate.opsForValue().set(  
    key,  
    inventory,  
    CACHE_TTL  
);
```

Explanation

The next request will be served directly from Redis.

This avoids another database query.

Logs

```
Saving Laptop into Redis  
  
TTL = 10 Minutes
```

Step 6 : Return Inventory

After caching the inventory, return the response to the client.

Code

```
return inventory;
```

Flow

```
PostgreSQL  
  |  
Inventory Service  
  |  
Redis Updated
```

|
Client

Step 7 : Reserve Inventory

When Saga sends a ReserveInventoryCommand, Inventory Service again checks Redis first.

Code

```
Inventory inventory =  
    cacheService.getInventory(  
        command.getProduct());
```

Explanation

Inventory reservation also follows the Cache Aside Pattern.

Redis is always checked before accessing PostgreSQL.

Step 8 : Reduce Stock

Decrease the available inventory.

Code

```
inventory.setStock(  
    inventory.getStock()  
    - command.getQuantity());
```

Example

Before

Laptop = 100

After

```
Laptop = 99
```

Step 9 : Update PostgreSQL

Persist the latest inventory into PostgreSQL.

Code

```
Inventory updatedInventory =  
    repository.save(inventory);
```

Database

```
inventory  
  
-----  
  
Laptop  
  
99
```

Explanation

PostgreSQL remains the primary source of truth.

Step 10 : Update Redis Cache

Update Redis immediately after successfully updating PostgreSQL.

Code

```
cacheService.updateCache(updatedInventory);
```

Inside InventoryCacheService

```
@CachePut(  
    value = "inventory",
```

```
        key = "#inventory.product"
    )
    public Inventory updateCache(
        Inventory inventory) {

        return inventory;
    }
}
```

Logs

```
=====

Redis Cache Updated

Product : Laptop

Stock : 99

=====
```

Flow

```
ReserveInventoryCommand
|
Inventory Service
|
Check Redis
|
CACHE HIT
|
Reduce Stock
|
Update PostgreSQL
|
Update Redis
|
InventoryReservedEvent
```


Summary

First Request

Client

|

Inventory Service

|

Redis

|

CACHE MISS

|

PostgreSQL

|

Save to Redis

|

Return Response

Second Request

Client

|

Inventory Service

|

Redis

|

CACHE HIT

|

Return Response

Benefits

- Reduced PostgreSQL load
- Faster response time
- Improved scalability
- Lower latency
- Frequently accessed inventory remains in memory
- Better performance for high-traffic systems

Interview Questions

Why Redis?

Redis improves performance by storing frequently accessed data in memory.

Which caching pattern did you implement?

Cache Aside Pattern.

What happens on a Cache MISS?

Inventory is loaded from PostgreSQL, stored in Redis, and returned to the client.

Why update Redis after PostgreSQL?

To ensure the cache always contains the latest inventory data and to avoid stale reads.